# Abderrahmene Naceri

**Project Portfolio | FPGA, Embedded and Hardware Systems**

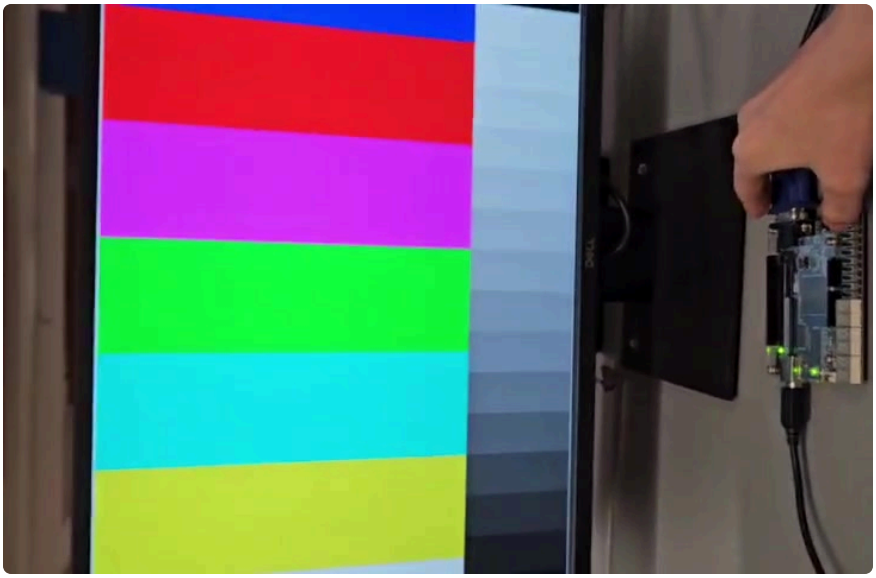
naceri@uwindsor.ca (226) 246-0994 linkedin.com/in/abderrahmene-naceri github.com/ANaceri26 Windsor, ON

## VGA Timing Controller and Frame Buffer

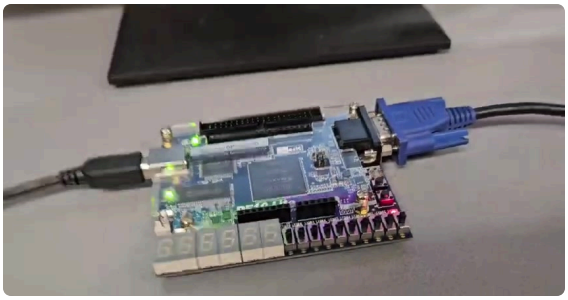
A Verilog video pipeline that draws to a 640×480 monitor from on-chip block RAM on a MAX 10 FPGA.

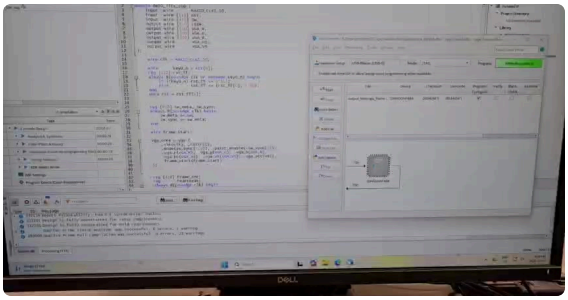
**Verilog**, Intel Quartus, ModelSim, Icarus Verilog, Terasic DE10-Lite (MAX 10)

Running on hardware

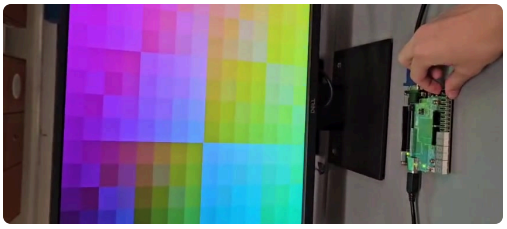


Test card on a Dell monitor: colour bars, grey ramp, 1-pixel white border.





| 640×480 @ 60 Hz | 921,600 bits | 307,200 pixels | 4 scenes |
|---|---|---|---|
| 800×525 total, 25 MHz pixel enable | dual-port block RAM frame buffer | checked per frame in simulation | switch-selected, plus freeze mode |



Gradient scene.

### Timing generator

Counters produce 640/16/96/48 horizontal and 480/10/2/33 vertical timing with active-low HSYNC/VSYNC.

25 MHz pixel enable from the 50 MHz clock keeps the whole design in one clock domain.

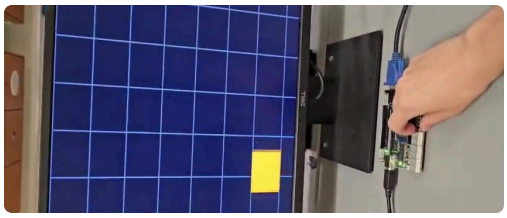
Reset and two-flop switch synchronizers; SDC constraints for the 50 MHz clock.

### Dual-port frame buffer

320×240 RGB444 buffer in M9K block RAM (76,800 × 12 bits), each pixel upscaled 2×2.

Painter writes during vertical blanking, staying ahead of the display read so frames never tear.

Address = row·256 + row·64 + column: shift-and-add instead of a multiplier.



Bouncing-box scene: moves 2 px right, 1 px down per frame.

```
stage 0  x, y, sync → RAM address
stage 1  RAM data valid | sync delayed
stage 2  colour + sync registered to pins
```

### Self-checking verification

Sync pipelined to match the RAM's one-cycle read latency, preventing a one-pixel image shift.

Testbench acts as a monitor: rebuilds position from sync pulses alone and checks line length, sync widths, blanking and all 307,200 visible pixels.

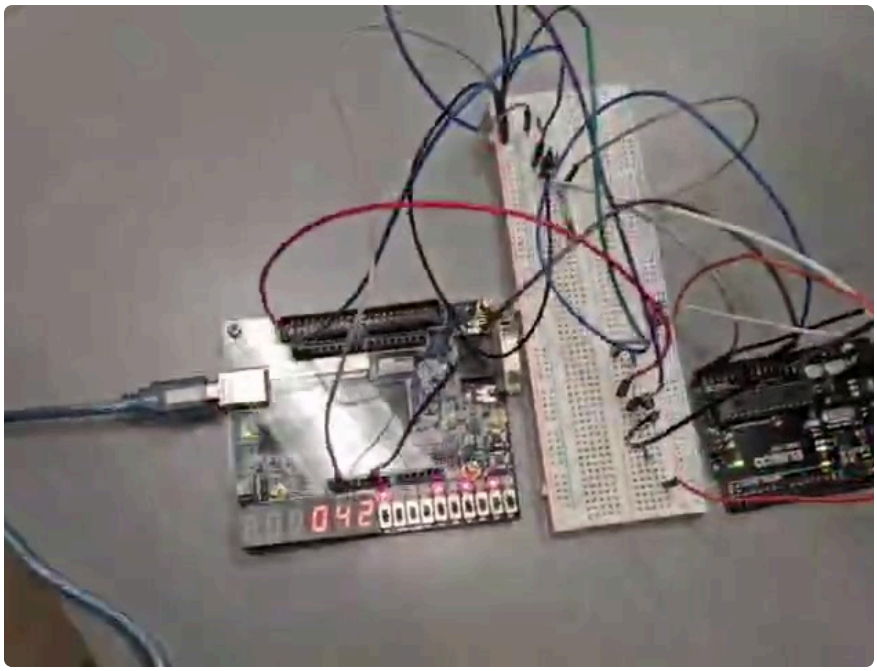
Six simulated frames (~5.8 M clock cycles) also cover animation and freeze.

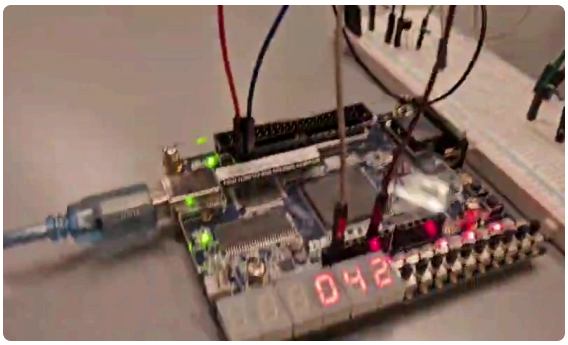
# FPGA-to-Arduino I2C Communication Link

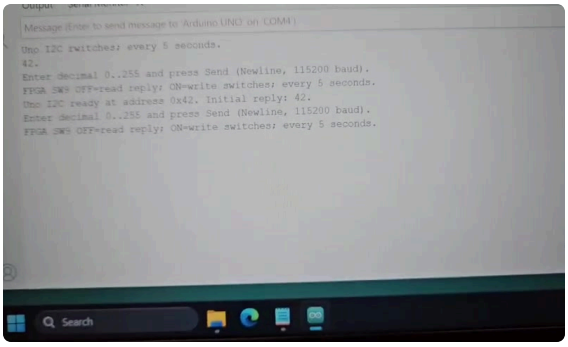
My own I2C controller, running on a MAX 10 FPGA, sending and receiving bytes with an Arduino Uno across a 3.3 V to 5 V bus.

**SystemVerilog**, C++ (Arduino), Intel Quartus, Python, DE10-Lite, Arduino Uno R3, BS170 MOSFETs

Running on hardware

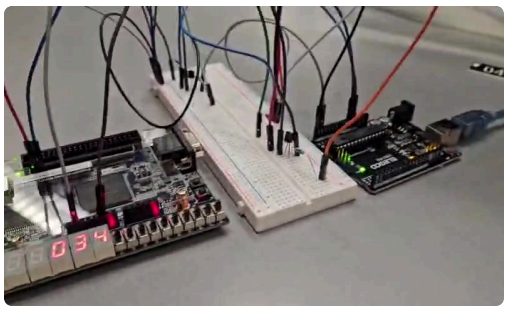







DE10-Lite (left) reading 042 from the Arduino Uno (right) through the breadboard level shifter.

| 10 kHz | Every 5 s | 72 pins | 12% / 18% |
|---|---|---|---|
| bus, FPGA controller, Uno at 0x42 | one read or write transaction | fitted pinout checked by script | Uno flash / RAM used |



## Hardware transaction scheduler

Issues START + address, one data byte and STOP, checking NACK, timeout and invalid-command flags at every stage.

Any error releases the bus with STOP; LED8 flashes until the next good transfer.

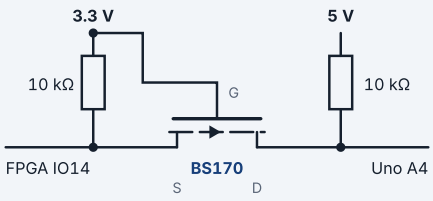
SW9 picks read or write, SW0–SW7 set the byte; switches synchronized and 20 ms debounced; result shown in binary on LEDs and 000–255 on 7-segment.

## 3.3 V to 5 V level shifting

One BS170 MOSFET per line, gate tied to 3.3 V, 10 kΩ pull-ups on each side.

FPGA IO14/IO15 (pins AB21/AA20) to Uno A4/A5 on a common ground; AVR internal pull-ups disabled.

Bus run at 10 kHz for rise-time margin through the shifter.




Same circuit on SCL (IO15 to A5)

```
Critical Warning 169085
72 pins with no exact location
clock: M8 → P11
SDA: A5 → AB21   SCL: C8 → AA20
```

## Firmware and board bring-up

Uno firmware uses Wire onReceive/onRequest interrupt callbacks with atomic copies of shared state and range-checked serial input.

Found Quartus had silently ignored all 72 pin assignments (144 constraints) due to QSF syntax; fixed the QSF/SDC.

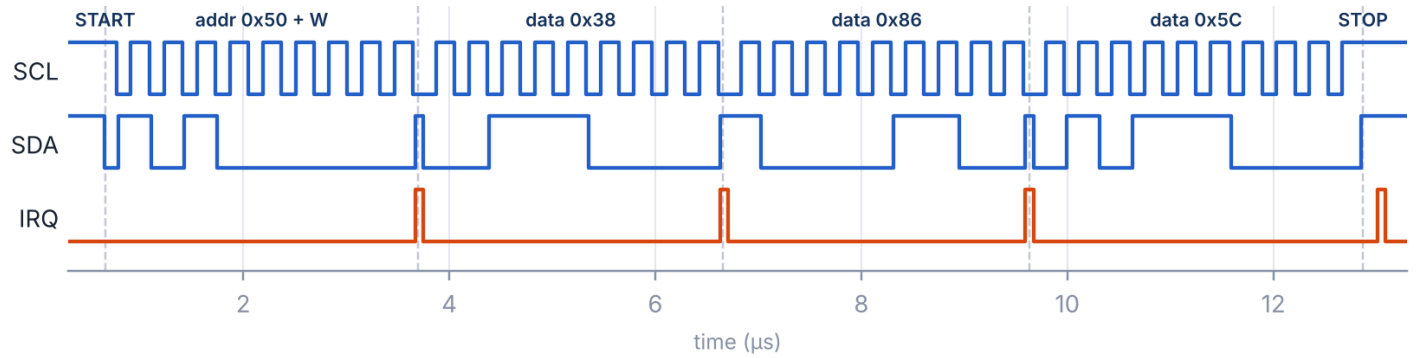
Wrote a Python checker that compares every fitted pin and I/O standard against the intended pinout.

# I2C Controller and Bare-Metal Firmware Driver

The reusable controller core behind the Arduino link, with a C driver tested directly against the RTL.

**SystemVerilog**, C11, Verilator, Icarus Verilog, Yosys, Python

Verified in simulation




Icarus Verilog trace from the regression: one write (address 0x50 + 3 data bytes), with the interrupt firing after each byte.

### RTL

18-state FSM: START, repeated START, STOP, ACK/NACK, clock stretching, programmable SCL divider.

Two-flop input synchronizers, command watchdog, sticky status and IRQ.

### Driver

C11 HAL with volatile-pointer MMIO register access.

Offset-based I/O layer: the same driver runs on hardware or against Verilated RTL.

### Verification

1,000 random transactions and 9,010 commands per seed, 2 seeds, 0 failures.

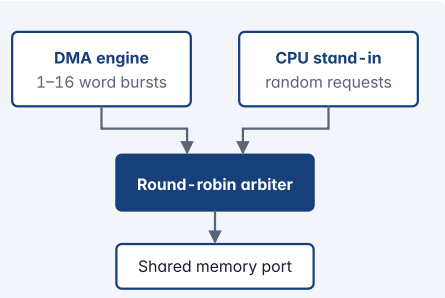
C/RTL co-sim: 100 write/read pairs (1,600 bytes) plus fault paths.

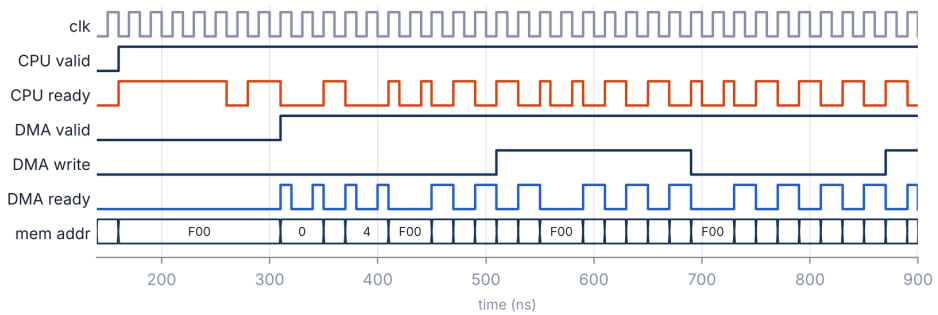
# DMA Controller with Round-Robin Bus Arbiter

A 32-bit memory-copy engine that shares one memory port fairly with a CPU.

**SystemVerilog**, C11, Verilator, Icarus Verilog, Yosys

Verified in simulation




Simulation trace: once the DMA starts, the CPU stand-in and the DMA alternate beats on the shared port.

### Engine

Memory-mapped CSRs, completed-word counter, IRQ, bus-error and timeout reporting.

Rejects misaligned, overlapping and 32-bit-overflow copies in hardware.

### Arbiter

Fair beat-level round robin; holds a grant across stalls so data stays stable under backpressure.

260,000+ contended bus beats verified.

### Verification

1,000 random copies per seed; 1, 8 and 16-word bursts.

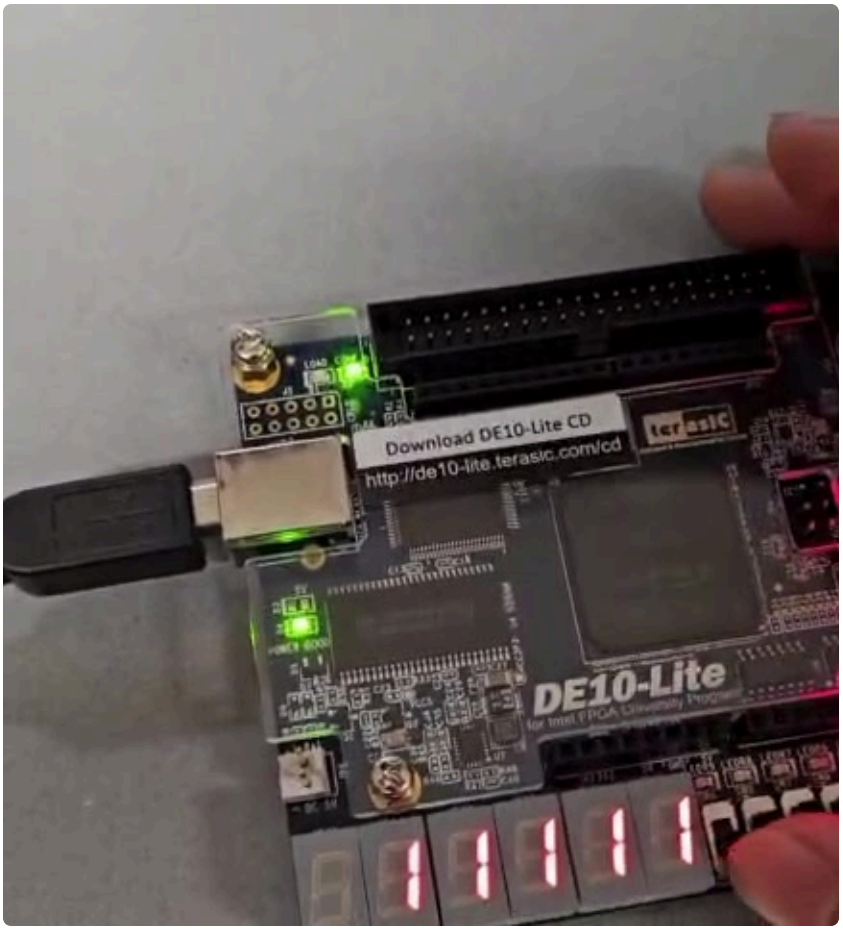
C driver co-simulated over 500 random copies; 64-word on-chip board self-test.

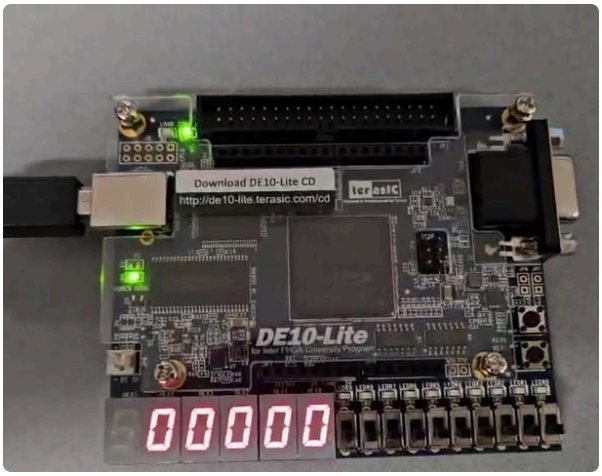
# 4-Bit Ripple-Carry and Carry-Lookahead Adders

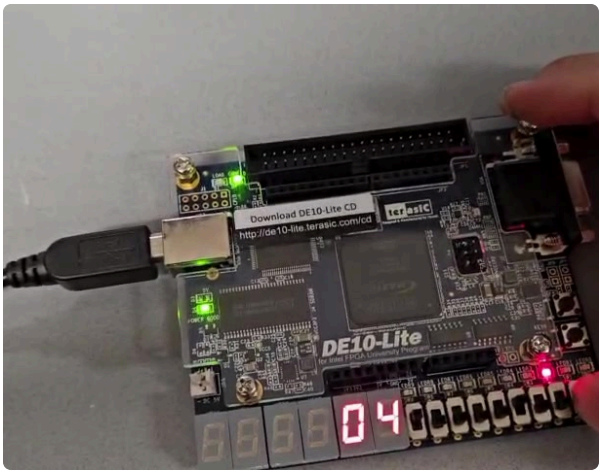
Two adder architectures written gate by gate in Verilog, with the ripple-carry design running on a DE10-Lite.

**Verilog**, Intel Quartus Prime 23.1, Questa, DE10-Lite (MAX 10 10M50DAF484C7G)

Running on hardware




Binary mode: 1111 + 1111 + carry-in = 11111 (31) on HEX4–HEX0 and the LEDs.

$$G_i = A_i \cdot B_i \qquad P_i = A_i \oplus B_i$$

$$C1 = G0 + P0 \cdot C0$$

$$C2 = G1 + P1 \cdot G0 + P1 \cdot P0 \cdot C0$$

$$C3 = G2 + P2 \cdot G1 + P2 \cdot P1 \cdot G0 + P2 \cdot P1 \cdot P0 \cdot C0$$

$$C4 = G3 + P3 \cdot G2 + P3 \cdot P2 \cdot G1 + P3 \cdot P2 \cdot P1 \cdot G0 + P3 \cdot P2 \cdot P1 \cdot P0 \cdot C0$$

## Carry-lookahead logic

RCA built from chained half-adder stages (XOR/AND/OR), so each carry waits for the one before it.

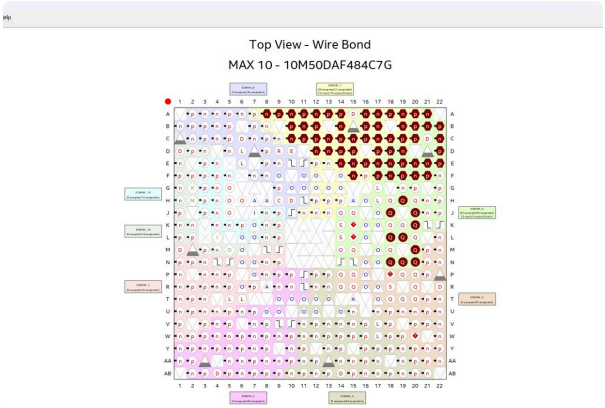
CLA expands every carry into generate/propagate terms of A, B and C0 only, removing the ripple chain whose delay grows linearly with width.

## Board implementation

SW0–SW3 = A, SW4–SW7 = B, SW8 = carry-in; sum and carry-out on LEDR0–LEDR4.

SW9 switches the 7-segment display between a per-bit binary readout and decimal 0–31 using combinational tens/ones conversion.

All pins mapped in Pin Planner; Questa gate-level netlist generated.



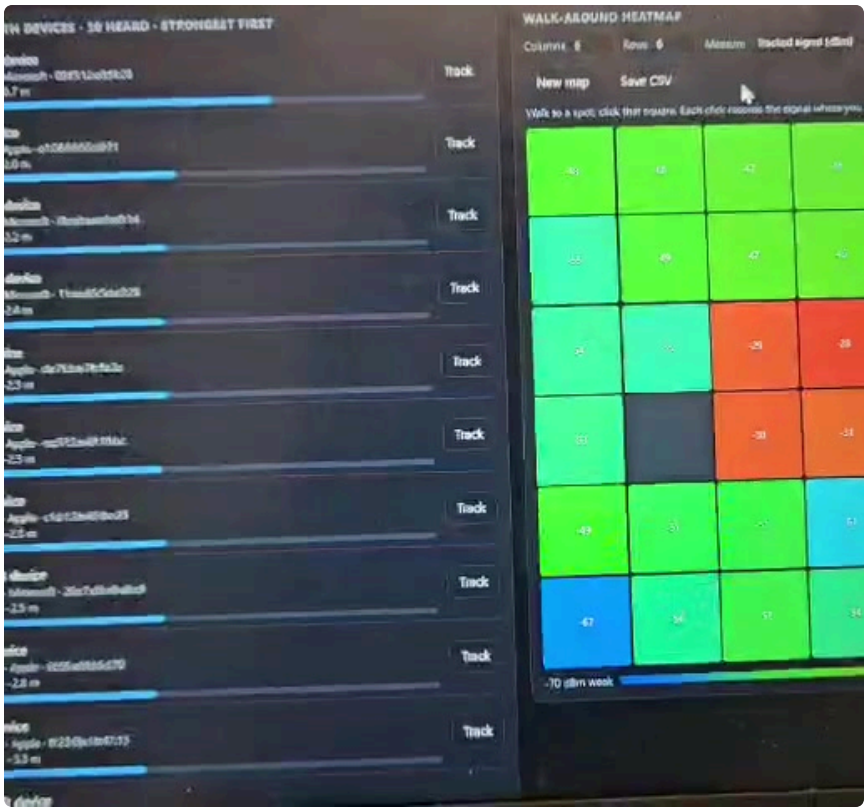

Quartus Pin Planner, MAX 10 10M50DAF484C7G.

# ESP32 Bluetooth and Wi-Fi Signal Finder

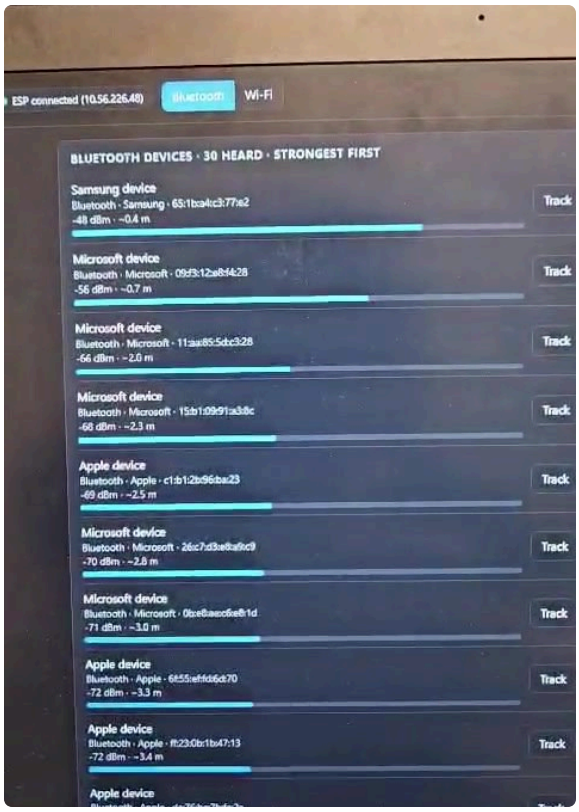
A handheld "metal detector" for wireless devices: it beeps faster as you get closer and maps signal strength as you walk.

**C++**, Python, NimBLE, UDP, JavaScript, Seeed XIAO ESP32-S3

Running on hardware

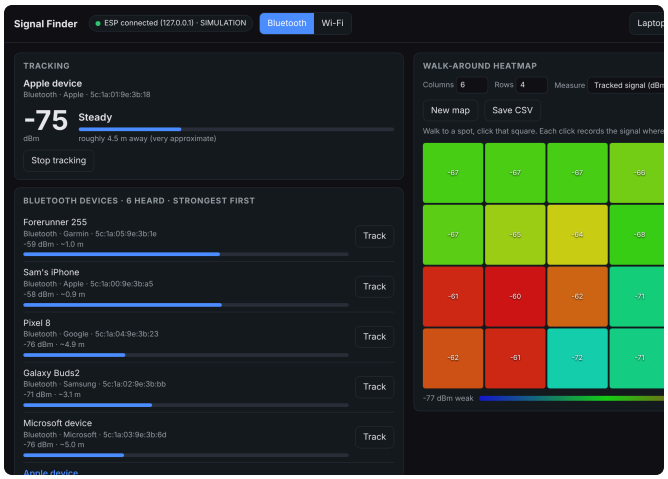



Live walk-around heatmap built from the ESP32's real Bluetooth scans.




Bluetooth devices heard by the board, strongest first.

| | | | |
|---|---|---|---|
| **80 devices** tracked at once, BLE or Wi-Fi | **300 ms** UDP report interval to the laptop | **α = 0.3** exponential RSSI smoothing | **400 ms** single-channel Wi-Fi rescan vs 4 s sweep |




Laptop app; simulation mode lets it run without the board.

## Laptop app

Dependency-free Python: threaded HTTP server and browser UI with a live signal list.

Track any device: the laptop beeps faster and higher as you walk closer.

Click-to-record heatmap of signal strength around a room, exportable to CSV.

## On the board

Scans BLE advertisements and Wi-Fi beacons; stale devices expire after 20 s.

Sends the 30 strongest signals over UDP, with a second port for commands.

## Standalone feedback

Buzzer rate and pitch rise with signal strength, even out of laptop range.

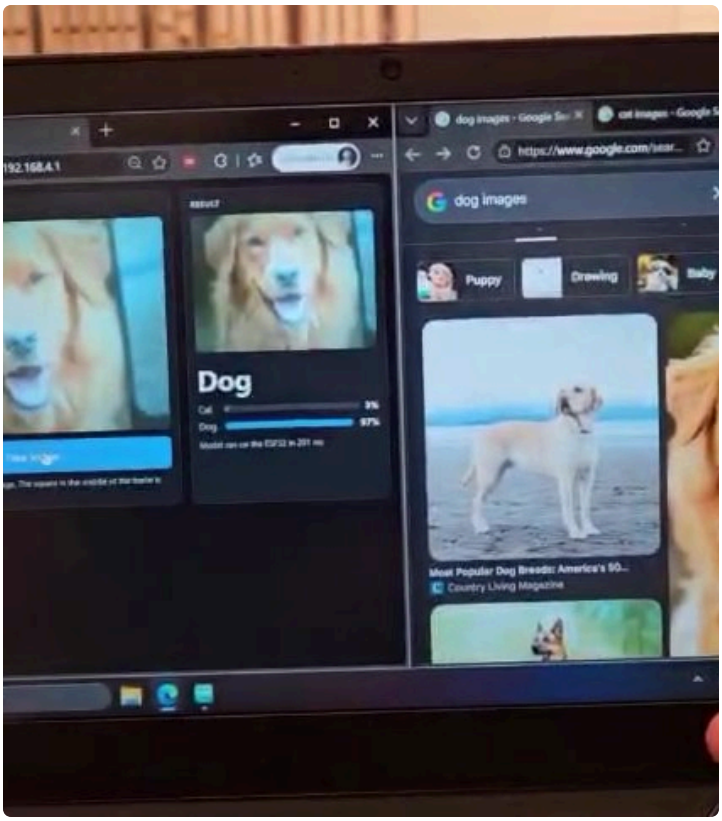
BOOT button switches BLE/Wi-Fi mode or locks onto the strongest signal.

# ESP32-S3 Edge AI Cat/Dog Classifier

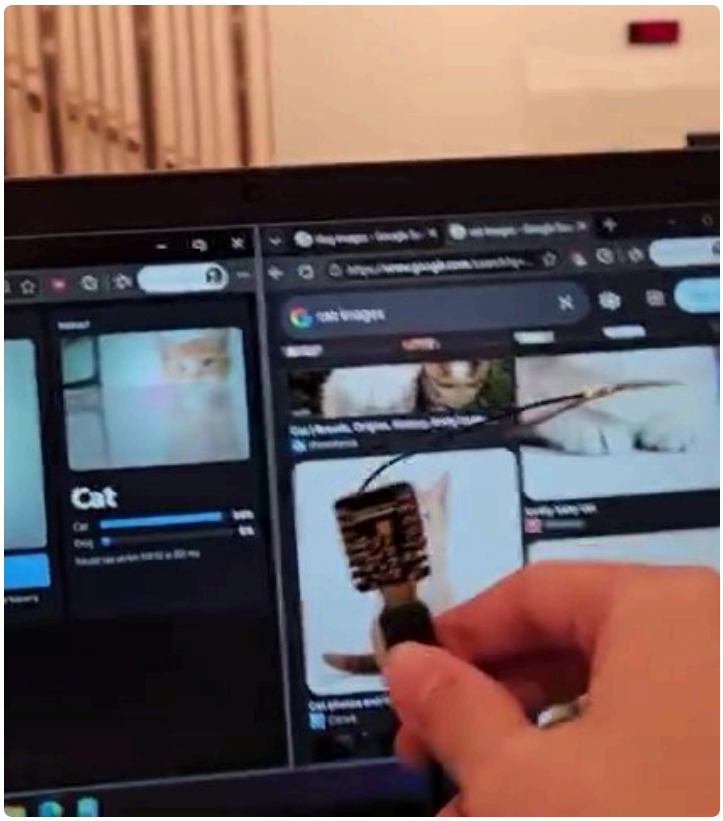
A MobileNetV2 image classifier trained on a laptop, shrunk to int8 and running on a thumb-sized camera board.

**Python**, TensorFlow, TensorFlow Lite Micro, C++, Seeed XIAO ESP32-S3 Sense

Running on hardware




The XIAO camera pointed at a dog photo; the board's web page reports "Dog".




Same board on a cat photo: "Cat".

| 23,262 images | 96×96 int8 | 1 MB | Every 1.5 s |
|---|---|---|---|
| Cats vs Dogs, 80/10/10 split | MobileNetV2, width 0.35 | PSRAM tensor arena | new prediction on device |

**Dataset**
Flip, rotation, zoom, brightness and contrast augmentation

→

**Transfer learning**
New head at 1e-3, then top 40 layers at 1e-5 with BatchNorm frozen

→

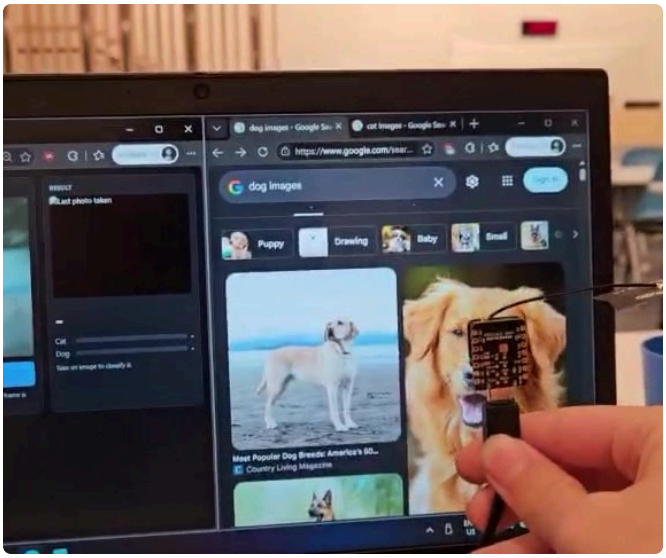
**int8 quantization**
300-image calibration set; accuracy re-checked with the int8 interpreter

→

**Firmware**
Model compiled into flash as a C array; TFLite Micro with ESP-NN

→

**On device**
320×240 frame, center-crop, 96×96, result to a phone page




## On the board

Camera frames are center-cropped exactly as in training, so the model sees the same input on the laptop and the chip.

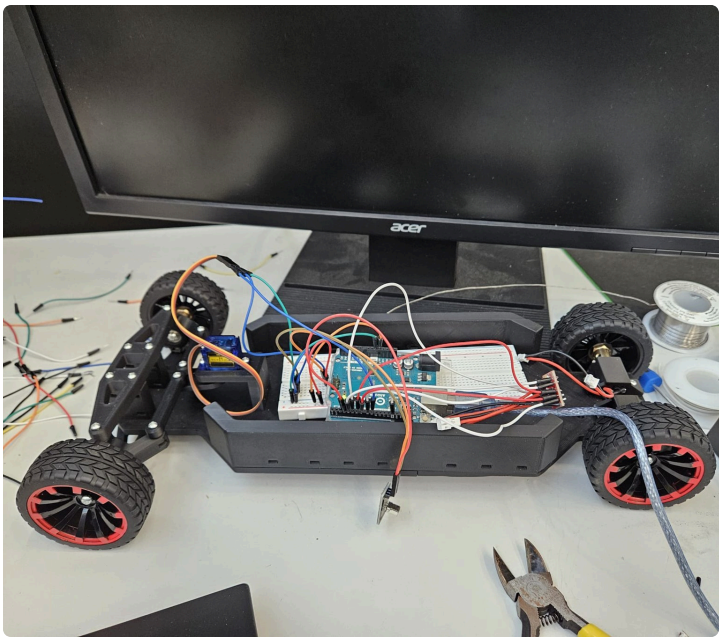
Predictions and live frames are served over the board's own Wi-Fi access point; under 70% confidence it reports "not sure".

The onboard LED lights when a dog is detected.

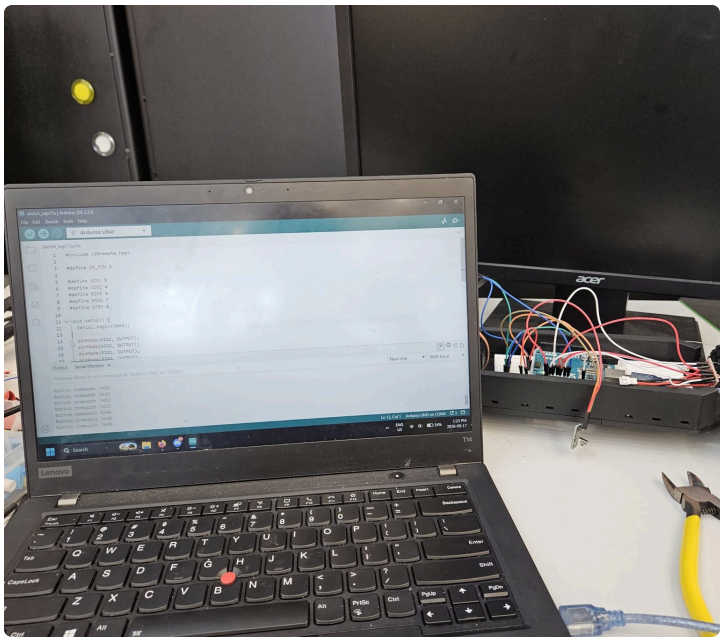
# Solar-Powered Vehicle

A two-motor rear-wheel-drive prototype charged from two solar panels.

**DC motors**, dual H-bridge driver, solar charge controller, batteries, Arduino Uno



Chassis with servo steering, driver and Uno on the breadboard.



Bench test: IR remote commands decoded on the Uno over serial.

### Power and drive

Bench-tested both motors through a dual H-bridge on independent channels before permanent wiring.

Integrated two solar panels through a charge controller to the battery; verified charger output under flashlight and sunlight.

Soldered permanent power and control wiring, driver-enable logic and dual-battery integration.

# Ubuntu Home Server and Personal Cloud

A Raspberry Pi that hosts my media, my files, DNS ad blocking and a VPN back home.

**Linux**, Raspberry Pi, Ubuntu Server, Bash, DNS, VPN, Plex



Raspberry Pi with the external drive used for cloud storage.

### Services

Always-on server administered over the Linux CLI and shell scripts, streaming movies and music with Plex.

Personal cloud storage on a drive attached to the Pi, replacing a paid cloud plan.

The Pi is the network's DNS server on a static IP, blocking ads and trackers for every device with no per-device software.

Self-hosted VPN to reach the home network from anywhere, carrying ad blocking and file access with it.

# Autonomous Fertilizer Robot

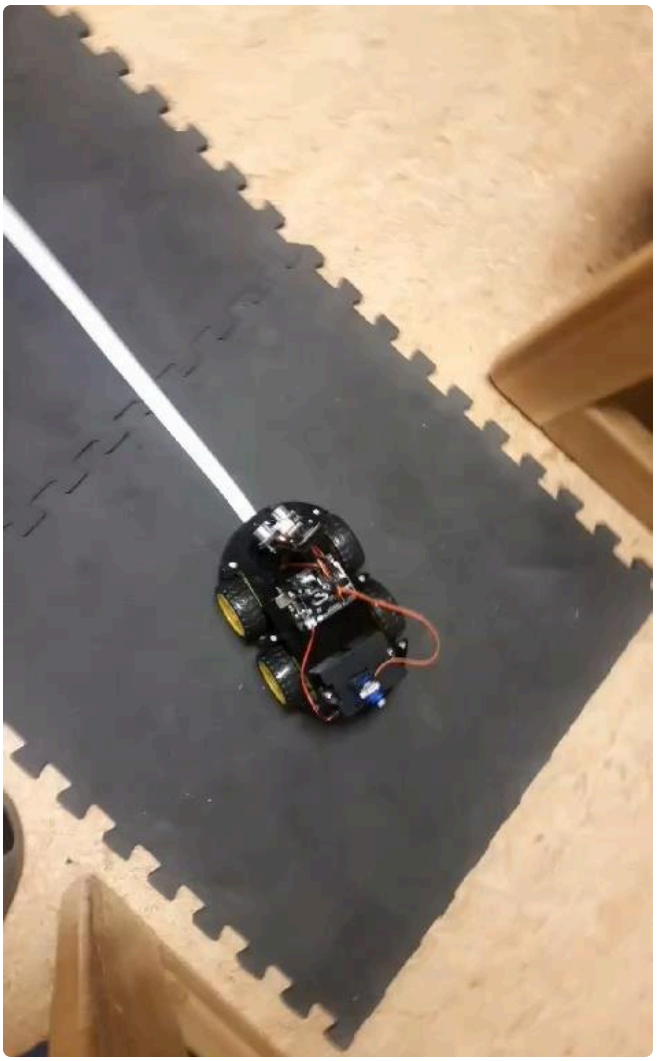
An ELEGOO Smart Robot Car that follows a course, avoids obstacles and drops fertilizer, built to compete.

**C++**, Arduino Uno R3 (ATmega328P), ultrasonic and line-tracking sensors, AutoCAD, LEGO

Competition finalist



Robot with its fertilizer dispenser.



Test run on the course before the competition.

| Under 40 s | $32.45 | Finals |
|---|---|---|
| full course time | build cost | competition result |



## Navigation and design

Line tracking keeps the car on the course; ultrasonic ranging detects and avoids obstacles.

Code tightened to fit the ATmega328P's limited memory for the finals.

LEGO-based dispenser is easy to modify; AutoCAD used to plan weight balance and fertilizer delivery.

# Interstellar Highway

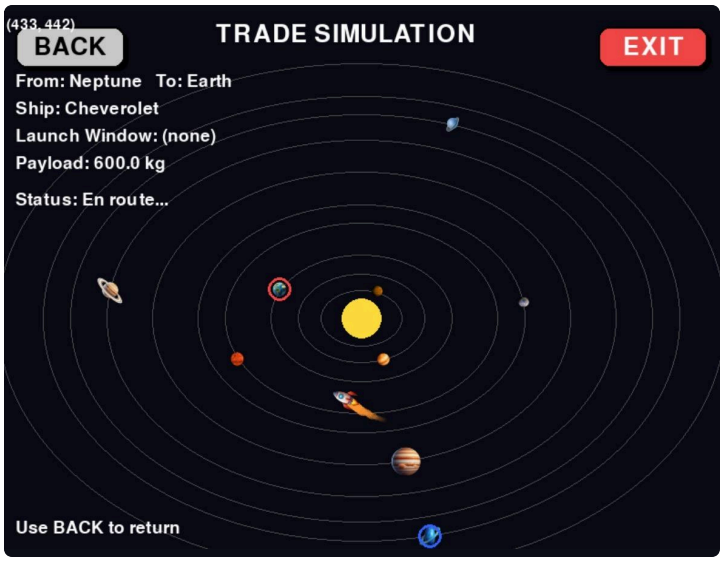
An interplanetary trade-route planner and animated simulator, built for the Ontario Engineering Competition.

**Python**, JSON, orbital mechanics (Lambert's problem, Tsiolkovsky rocket equation)

Ontario Engineering Competition




Title screen with mission and instruction modes.




Trade simulation: a ship en route from Neptune to Earth with a 600 kg payload.

| 10 bodies | 7 ship classes | 83% | CLI + GUI |
|---|---|---|---|
| transfer orbits between them | delta-v and fuel per class | trajectory accuracy vs theory | routes returned as JSON |



Our team at the competition.

## Route planning

Lambert's problem solvers compute transfer orbits between 10 celestial bodies.

Tsiolkovsky rocket equation gives delta-v and fuel for 7 spacecraft classes, optimizing launch windows and payload mass.

Ranks trade routes by efficiency and returns flight data as JSON, including hyperbolic escape/insertion vectors and parking orbits.

## Competition work

Control simulations for orbital mechanics, timing thruster "actuators".

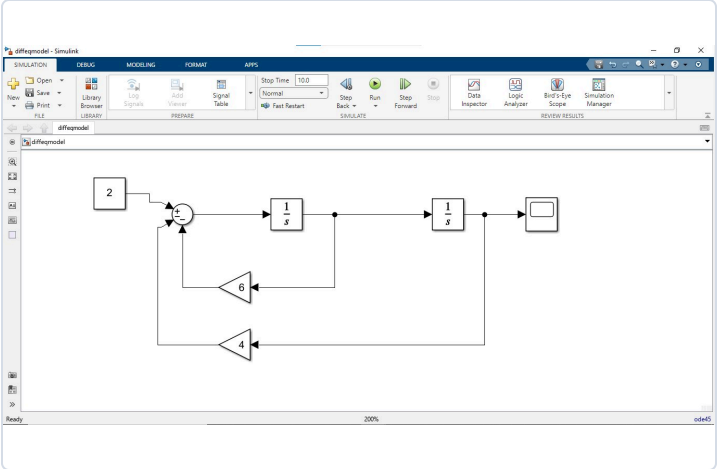
Optimized power budgets under strict constraints.

Trajectory model matched theoretical orbital physics with 83% accuracy.

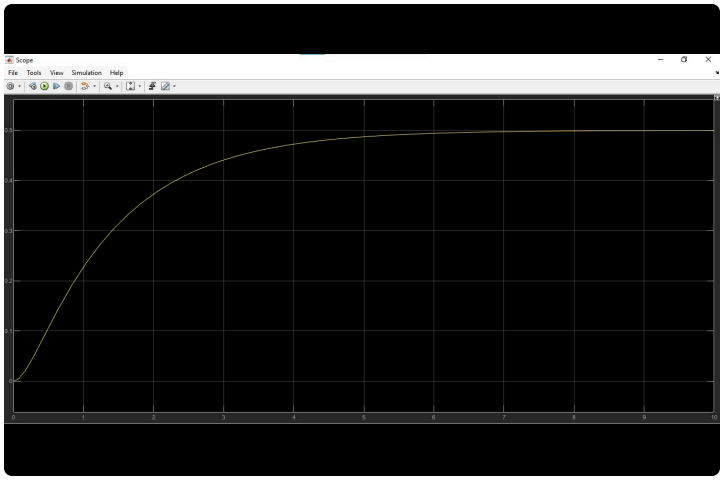
# Dynamic Systems Simulation

Second-order systems modelled in Simulink and checked against the hand-solved answer.

**MATLAB**, Simulink (MathWorks MATLAB and Simulink Onramp, 100%)




Integrator and feedback-gain model of y'' + 6y' + 4y = 2.



Scope output settling to the predicted steady state.

```
y'' + 6y' + 4y = 2
steady state: y'' = y' = 0
4y = 2 → y = 0.5
```

## Method

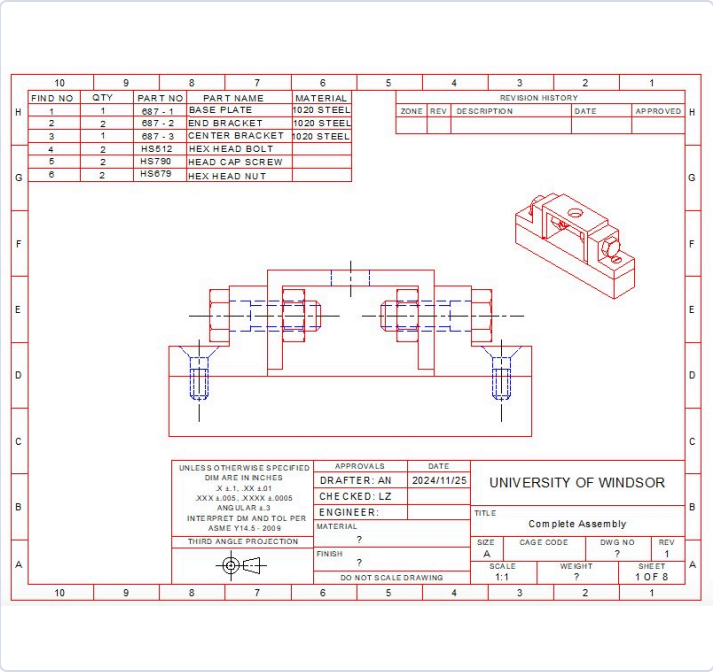
Governing equation built from integrator blocks and feedback loops.

Tuned damping ratio and feedback gains to study stability; simulated steady state matched the analytical value.

# Mechanical Assembly Design

A full drawing set for a 1020 steel bracket assembly to ASME Y14.5.

**AutoCAD**, GD&T (ASME Y14.5), third-angle projection



| FIND NO | QTY | PART NO | PART NAME | MATERIAL |
|---|---|---|---|---|
| 1 | 1 | 687 - 1 | BASE PLATE | 1020 STEEL |
| 2 | 2 | 687 - 2 | END BRACKET | 1020 STEEL |
| 3 | 1 | 687 - 3 | CENTER BRACKET | 1020 STEEL |
| 4 | 2 | HS512 | HEX HEAD BOLT | |
| 5 | 2 | HS790 | HEAD CAP SCREW | |
| 6 | 2 | HS679 | HEX HEAD NUT | |

| REVISION HISTORY | | | | |
|---|---|---|---|---|
| ZONE | REV | DESCRIPTION | DATE | APPROVED |

UNLESS OTHERWISE SPECIFIED DIM ARE IN INCHES .X ±.1, .XX ±.01 .XXX ±.005, .XXXX ±.0005 ANGULAR ±.3 INTERPRET DM AND TOL PER ASME Y14.5 - 2009 THIRD ANGLE PROJECTION

| APPROVALS | DATE |
|---|---|
| DRAFTER: AN | 2024/11/25 |
| CHECKED: LZ | |
| ENGINEER: | |

MATERIAL: ?
FINISH: ?
DO NOT SCALE DRAWING

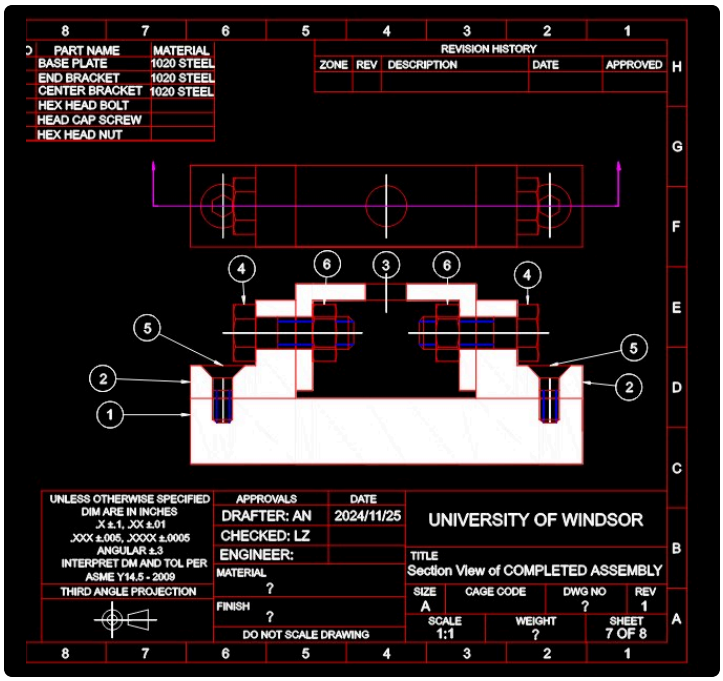
UNIVERSITY OF WINDSOR

TITLE: Complete Assembly

| SIZE | CAGE CODE | DWG NO | REV |
|---|---|---|---|
| A | | ? | 1 |

| SCALE | WEIGHT | SHEET |
|---|---|---|
| 1:1 | ? | 1 OF 8 |

Complete assembly with bill of materials.




Section view of the completed assembly.

## Drawing set

Orthographic projections, isometric and section views, standard parts sheet and BOM (base plate, end and center brackets, fasteners).

Toleranced with GD&T so parts fit and can be manufactured.

# ICTC CyberTitan Competition

A province-wide defence competition: keep a simulated lunar base running while it is under cyberattack.

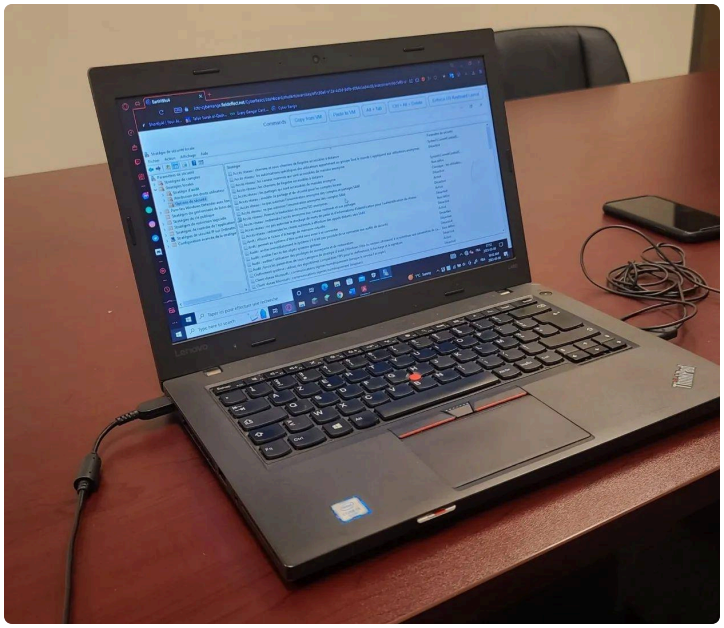
**Windows Server**, Remote Desktop, Wireshark, WinPcap, Local Security Policy, firewall rules

3rd place, Ontario



Team hardening systems during the competition.





| 3rd place | 15+ | Windows |
|---|---|---|
| province-wide defence competition | vulnerabilities found and fixed | servers hardened under live attack |

## What we did

Audited system logs and firewall rules to find and fix 15+ vulnerabilities under time pressure.
Reviewed Windows Local Security Policy and firewall configuration to reduce the attack surface.

# HomeLab and Network Security Research

An isolated lab for practising attack and defence without touching real systems.

**Kali Linux**, Ubuntu, Nmap, Wireshark, Metasploit, virtual machines, Live USB

### Lab

Kali and Ubuntu VMs plus Live USB environments for repeatable, sandboxed testing.

Nmap host, port and service enumeration to test firewall rules and cut attack surface.

### Findings

Wireshark traffic analysis and Metasploit assessments to study exploit chains and fixes.

Found misconfigurations in legacy routing hardware and hardened it by correcting configs and closing ports.

# Software Projects

Smaller builds in Python and Excel/VBA.

## Secure Login System

Python, Tkinter, CSV, hashlib

- SHA-256 password hashing with CSV-based user storage.
- 30-second lockout after 3 failed attempts to stop brute forcing.
- GUI with light/dark themes and frame switching.

## Automatic Grades Calculator

Excel, VBA

- VBA macros automate weighted-percentage grading for teachers.
- Scripts import attendance and student data and create a worksheet per class.